Université Cadi Ayyad
École Nationale des Sciences Appliquées de Safi
Cycle ingénieur, année universitaire 2026-2027

Module : Théorie des Jeux et Systèmes Multi-Agents
Projet intégrateur n° 6

Dilemme du prisonnier répété : tournoi d'Axelrod et agents LLM

Mini-livrable de la semaine 1 : cadrage et théorie

Séance du 11 novembre 2026

Réalisé par (groupe 6) :
TSETSE Komla Edem Wilfried (IDIA3)
SADOUR Assia (IDIA3)
MARZAQ Jihane
ZERBAOUI Marwane

Encadré par :
Pr Abdelilah JRAIFI

1. Objet du projet

Le projet n° 6 étudie le dilemme du prisonnier répété. Nous simulons un tournoi entre six stratégies (toujours coopérer, toujours trahir, tit-for-tat, rancunier, Pavlov, aléatoire), mesurons l'effet du bruit et l'évolution d'une population par la dynamique du réplicateur, puis faisons jouer des agents LLM locaux avec Ollama pour comparer leur comportement aux stratégies classiques. Le lien avec le cours est un paradoxe : à horizon fini et connu, l'induction à rebours prédit la trahison à chaque manche, alors que les stratégies conditionnelles coopèrent en pratique.

2. Travail réalisé

2.1 Organisation de l'équipe

Le groupe compte quatre membres, un rôle chacun, comme le prévoit le guide. La répartition vaut jusqu'à la fin de la semaine 4, puis les rôles tournent d'un cran. Un présentateur différent est prévu chaque semaine.

Membre	Rôle (semaines 1 à 4)	Rôle après la rotation	Présente en
Wilfried	Chef de projet et théorie	Quant / Python	S1, S5
Assia	Quant / Python	Développeuse de l'agent	S2, S6
Marwane	Développeur de l'agent	Testeur	S3, S7
Jihane	Testeuse	Cheffe de projet	S4

2.2 Dépôt GitHub

Le dépôt privé du groupe (github.com/wekt2k04/projet-groupe6-dilemme-repete-axelrod) est créé et organisé selon l'annexe A du guide : src, tests, notebooks, docs, requirements. Les trois autres membres ont reçu une invitation avec droit d'écriture ; leur premier commit nominatif suivra l'acceptation. Chaque modification passera par une branche et une pull request relue par un autre membre.

2.3 Environnement technique

Le projet tourne sous Python 3.13 dans un environnement virtuel (NumPy, SciPy, pandas, pytest, LangChain 1.5 avec langchain-ollama). Ollama est installé sur le poste du chef de projet, configuré pour n'écouter que sur 127.0.0.1, avec les modèles llama3.1 (4,9 Go) et qwen2.5 (4,7 Go) téléchargés. L'installation reste à reproduire chez les autres membres.

2.4 Fiche de lecture et théorie

La fiche de lecture de deux pages pose le jeu de base, $T = 5$, $R = 3$, $P = 1$, $S = 0$, et la condition du dilemme $T > R > P > S$ avec $2R > T + S$ (ici $6 > 5$). Trahir domine strictement, donc (D, D) est l'unique équilibre de Nash du jeu à une manche, alors que (C, C) rapporte 3 à chacun au lieu de 1. À horizon indéterminé de facteur d'actualisation δ, le rancunier soutient la coopération si $\delta \geq (T - R)/(T - P) = 1/2$. Pour tit-for-tat, la déviation qui alterne trahir et coopérer exige $\delta \geq (T - R)/(R - S) = 2/3$. La dynamique du réplicateur met à jour les parts par $x_i(t+1) = x_i(t)\, f_i / \bar{f}$.

2.5 Prototype Python et premier tournoi

Un premier module jeux.py implémente les six stratégies, le tournoi à toutes les paires (auto-jeu compris), le bruit et le réplicateur ; huit tests pytest passent (TFT contre TFT donne (3 ; 3), toujours trahir contre toujours coopérer (5 ; 0), TFT ne perd que la première manche contre toujours trahir, parts du réplicateur

positives de somme 1, seuil 1/2 du rancunier). Un tournoi exploratoire de 200 manches (graine 42, 5 répétitions, score moyen par manche) classe sans bruit le rancunier (2,67), TFT (2,54), Pavlov (2,45), toujours trahir (2,36), toujours coopérer (2,27) et l'aléatoire (1,96). Avec un bruit de 5 %, toujours trahir (2,39) et le rancunier (2,35) passent devant Pavlov (2,08), l'aléatoire (2,00), TFT (1,97) et toujours coopérer (1,64) : la domination de TFT ne résiste pas au bruit dans ce premier essai, à confirmer sur plus de graines.

2.6 Planification

Le diagramme de Gantt prévisionnel couvre les sept semaines. Il précise le responsable de chaque tâche, les jalons hebdomadaires et la rotation des rôles à la fin de la semaine 4 (voir annexe). Le classeur gantt_source.xlsx est la source éditable, avec une grille prévue et une grille réalisée.

3. Choix retenus

Trois conventions structurent la suite. Les scores sont toujours des gains moyens par manche, ce qui permet de comparer un LLM qui joue 10 manches à une stratégie qui en joue 200. Le bruit inverse chaque coup de chacun des deux joueurs avec la probabilité ε, et les stratégies voient les coups exécutés. Dans le tournoi, une stratégie joue aussi contre elle-même et son score est la moyenne des deux côtés. Le calcul reste écrit en fonctions pures, sans LangChain, testées séparément ; les outils de l'agent se contentent de les appeler, avec des paramètres plafonnés et jamais d'eval ni d'exec sur un texte produit par un modèle.

4. Écarts, difficultés et questions à l'encadrant

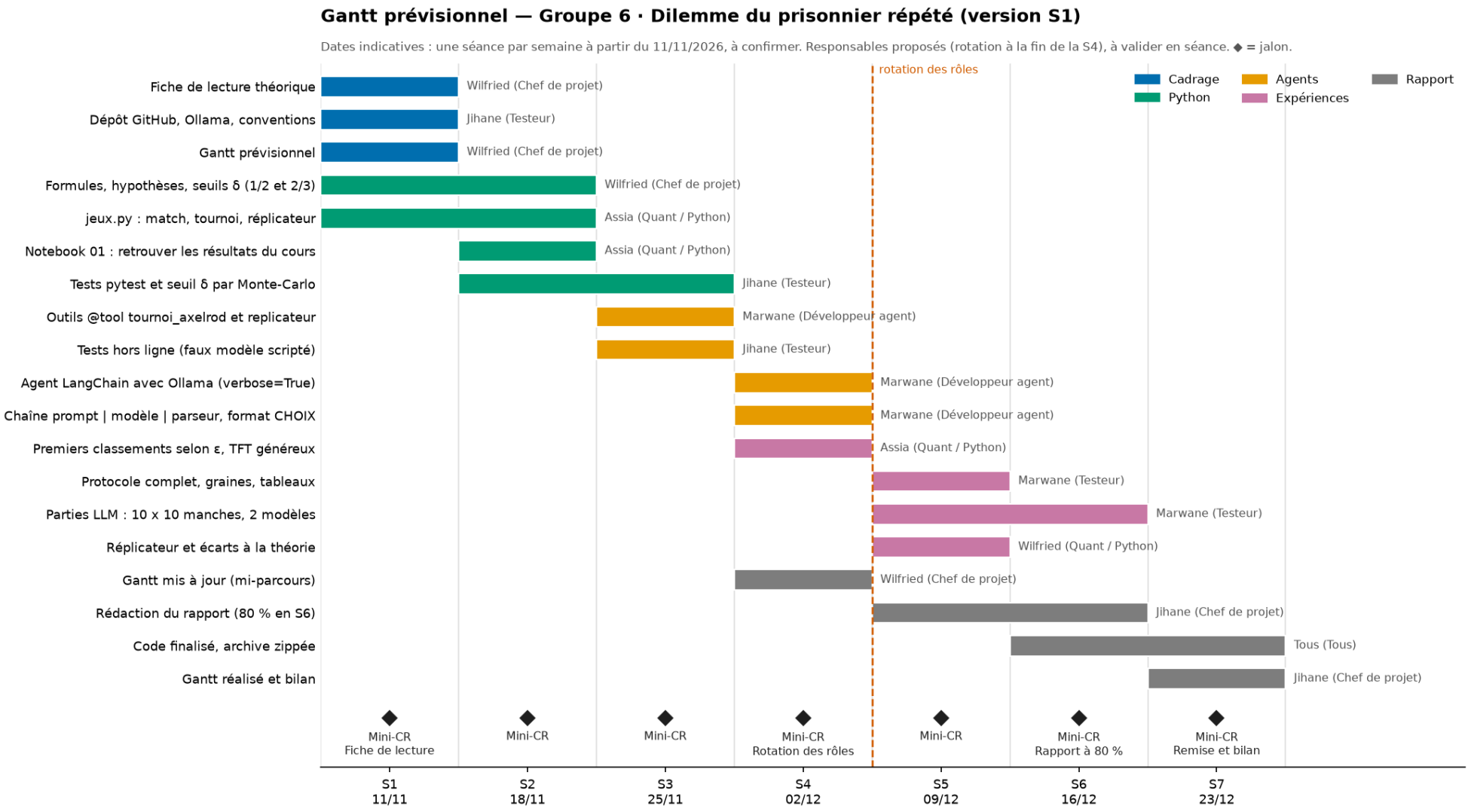
Prévu pour la semaine 1 : dépôt, Ollama avec llama3.1, fiche de lecture, fonctions du cours à étendre, Gantt prévisionnel. Réalisé : fiche, dépôt, Gantt et Ollama sont en place sur le poste du chef de projet. Un prototype de jeux.py et ses tests sont en avance sur le planning, mais la tâche reste en cours jusqu'en S2 car les conventions doivent encore être validées. Restent à faire : l'installation d'Ollama chez les trois autres membres et leur premier commit. Difficulté : le code du cours (agent_tj_sma.py et ses outils) ne figure pas dans nos supports ; nous partons des conventions du guide (outils @tool, sortie JSON) et nous adapterons dès que nous l'aurons.

1. Quelle sera la date de la séance de la semaine 2, le 18 novembre étant un jour férié ?
2. Pouvez-vous nous transmettre agent_tj_sma.py, les modules de jeux et le chapitre 6 sur le superviseur multi-agents ?
3. Dans les parties LLM, l'adversaire d'un agent est-il une stratégie classique comme TFT, ou un second agent LLM ?

5. Objectifs de la semaine 2

La semaine 2 est consacrée au prototype Python. Un notebook retrouvera les résultats du cours (dilemme à une manche, induction à rebours sur n manches), les formules seront expliquées, et le seuil $\delta = 1/2$ du rancunier sera vérifié numériquement par une partie à arrêt aléatoire. Les conventions seront figées, jeux.py complété et testé, chaque membre déposera au moins un commit et le Gantt réalisé sera mis à jour.

Annexe : diagramme de Gantt prévisionnel



Planning prévisionnel des semaines 1 à 7, avec les responsables de chaque tâche et la rotation des rôles en fin de semaine 4.